

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE CODE: ITA0606 COURSE NAME: MACHINE LEARNING**

### **COURSE OUTCOME COVERED**

***CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)***

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5 Total Marks:100 Annexure A**

### **Experiments**

#### ***Code of Conduct:***

*I Gayathri N P (192421362) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## Annexure A

1. **Design and conduct an experiment to implement a Naïve Bayes classifier on a real world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.**

### Program

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print("Dataset Shape:", X.shape)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train_1 = X_train.iloc[:, :5]
X_test_1 = X_test.iloc[:, :5]
model1 = GaussianNB()
model1.fit(X_train_1, y_train)
y_pred1 = model1.predict(X_test_1)
print("\n--- Feature Set 1: First 5 Features ---")
print("Accuracy :", accuracy_score(y_test, y_pred1))
print("Precision:", precision_score(y_test, y_pred1))
print("Recall  :", recall_score(y_test, y_pred1))

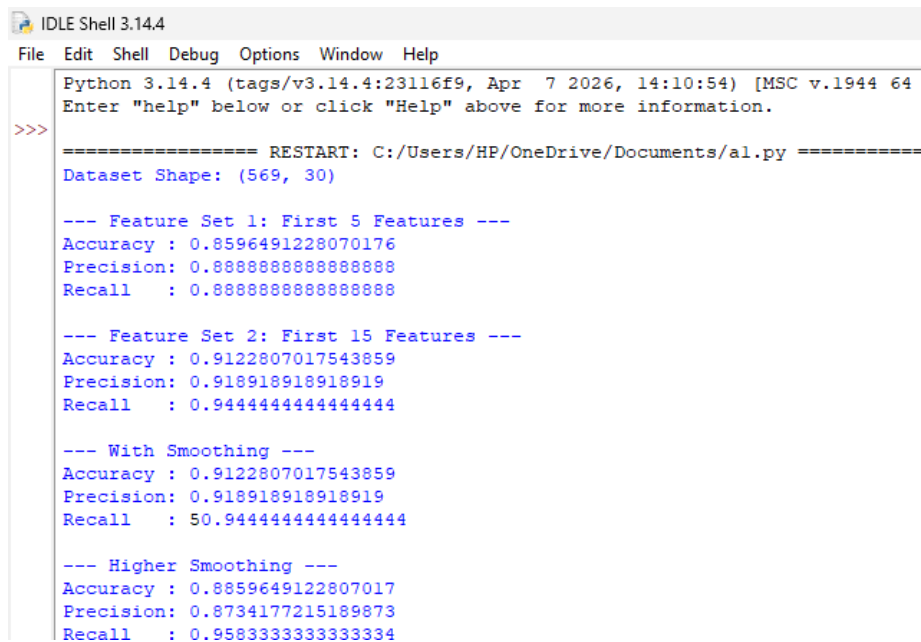
X_train_2 = X_train.iloc[:, :15]
X_test_2 = X_test.iloc[:, :15]
model2 = GaussianNB()
model2.fit(X_train_2, y_train)
y_pred2 = model2.predict(X_test_2)
print("\n--- Feature Set 2: First 15 Features ---")
print("Accuracy :", accuracy_score(y_test, y_pred2))
```

```

print("Precision:", precision_score(y_test, y_pred2))
print("Recall  :", recall_score(y_test, y_pred2))
model_smooth = GaussianNB(var_smoothing=1e-9)
model_smooth.fit(X_train_2, y_train)
y_pred_smooth = model_smooth.predict(X_test_2)
print("\n--- With Smoothing ---")
print("Accuracy :", accuracy_score(y_test, y_pred_smooth))
print("Precision:", precision_score(y_test, y_pred_smooth))
print("Recall  :", recall_score(y_test, y_pred_smooth))
model_smooth2 = GaussianNB(var_smoothing=1e-2)
model_smooth2.fit(X_train_2, y_train)
y_pred_smooth2 = model_smooth2.predict(X_test_2)
print("\n--- Higher Smoothing ---")
print("Accuracy :", accuracy_score(y_test, y_pred_smooth2))
print("Precision:", precision_score(y_test, y_pred_smooth2))
print("Recall  :", recall_score(y_test, y_pred_smooth2))

```

## Output



```

IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr  7 2026, 14:10:54) [MSC v.1944 64
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/HP/OneDrive/Documents/al.py =====
Dataset Shape: (569, 30)

--- Feature Set 1: First 5 Features ---
Accuracy : 0.8596491228070176
Precision: 0.8888888888888888
Recall   : 0.8888888888888888

--- Feature Set 2: First 15 Features ---
Accuracy : 0.9122807017543859
Precision: 0.918918918918919
Recall   : 0.9444444444444444

--- With Smoothing ---
Accuracy : 0.9122807017543859
Precision: 0.918918918918919
Recall   : 50.9444444444444444

--- Higher Smoothing ---
Accuracy : 0.8859649122807017
Precision: 0.8734177215189873
Recall   : 0.9583333333333334

```

2. **Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.**

**Program:**

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
data = np.random.normal(loc=50, scale=10, size=100)
mle_mean = np.mean(data)
mle_variance = np.mean((data - mle_mean) ** 2)
print("===== MLE ESTIMATION =====")
print("MLE Mean    :", mle_mean)
print("MLE Variance :", mle_variance)
prior_mean = 45
prior_variance = 25
observation_variance = 100
n = len(data)
sample_mean = np.mean(data)
posterior_variance = 1 / (
    (1 / prior_variance) +
    (n / observation_variance)
)
posterior_mean = posterior_variance * (
    (prior_mean / prior_variance) +
    (n * sample_mean / observation_variance)
)
print("\n===== BAYESIAN ESTIMATION =====")
print("Prior Mean      :", prior_mean)
print("Prior Variance   :", prior_variance)
print("Posterior Mean    :", posterior_mean)
print("Posterior Variance :", posterior_variance)
small_data = data[:10]
small_mean = np.mean(small_data)
small_posterior_variance = 1 / (
    (1 / prior_variance) +
```

```

        (len(small_data) / observation_variance)
    )
    small_posterior_mean = small_posterior_variance * (
        (prior_mean / prior_variance) +
        (len(small_data) * small_mean / observation_variance)
    )
    print("\n===== SMALL SAMPLE =====")
    print("Sample Size      :", len(small_data))
    print("MLE Mean          :", small_mean)
    print("Bayesian Mean      :", small_posterior_mean)
    large_data = data
    large_mean = np.mean(large_data)
    large_posterior_variance = 1 / (
        (1 / prior_variance) +
        (len(large_data) / observation_variance)
    )
    large_posterior_mean = large_posterior_variance * (
        (prior_mean / prior_variance) +
        (len(large_data) * large_mean / observation_variance)
    )
    print("\n===== LARGE SAMPLE =====")
    print("Sample Size      :", len(large_data))
    print("MLE Mean          :", large_mean)
    print("Bayesian Mean      :", large_posterior_mean)
    plt.hist(data, bins=15, edgecolor="black")
    plt.axvline(mle_mean, label="MLE Mean")
    plt.axvline(posterior_mean, label="Bayesian Mean")
    plt.xlabel("Value")
    plt.ylabel("Frequency")
    plt.title("MLE vs Bayesian Estimation")
    plt.legend()
    plt.show()

```

## Output

```
=====
===== MLE ESTIMATION =====
MLE Mean      : 48.96153482605907
MLE Variance  : 81.65221946938584

===== BAYESIAN ESTIMATION =====
Prior Mean    : 45
Prior Variance : 25
Posterior Mean : 48.80916810197987
Posterior Variance : 0.9615384615384615

===== SMALL SAMPLE =====
Sample Size   : 10
MLE Mean      : 54.48061111698756
Bayesian Mean : 51.77186508356254

===== LARGE SAMPLE =====
Sample Size   : 100
MLE Mean      : 48.96153482605907
Bayesian Mean : 48.80916810197987
```

3. **Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.**

### Program :

P\_Rain = 0.30

P\_NoRain = 0.70

P\_Wet\_Rain = 0.80

P\_Wet\_NoRain = 0.20

P\_Traffic\_Rain = 0.70

P\_Traffic\_NoRain = 0.30

print("===== WITHOUT EVIDENCE =====")

print("P(Rain) =", P\_Rain)

print("P(No Rain) =", P\_NoRain)

P\_WetGrass = (

    P\_Wet\_Rain \* P\_Rain

    +

    P\_Wet\_NoRain \* P\_NoRain

)

```

P_Rain_given_Wet = (
    P_Wet_Rain * P_Rain
) / P_WetGrass

```

```

print("\n===== EVIDENCE: WET GRASS =====")

```

```

print("P(Wet Grass) =", P_WetGrass)
print("P(Rain | Wet Grass) =", P_Rain_given_Wet)
P_Dry_Rain = 1 - P_Wet_Rain
P_Dry_NoRain = 1 - P_Wet_NoRain

```

```

P_DryGrass = (
    P_Dry_Rain * P_Rain
    +
    P_Dry_NoRain * P_NoRain
)

```

```

P_Rain_given_Dry = (
    P_Dry_Rain * P_Rain
) / P_DryGrass

```

```

print("\n===== EVIDENCE: DRY GRASS =====")

```

```

print("P(Dry Grass) =", P_DryGrass)
print("P(Rain | Dry Grass) =", P_Rain_given_Dry)
P_Traffic = (

```

```

    P_Traffic_Rain * P_Rain
    +
    P_Traffic_NoRain * P_NoRain

```

```

)
print("\n===== TRAFFIC =====")

```

```

print("P(Traffic) =", P_Traffic)

```

```

print("\n===== AFTER MODIFYING PROBABILITY =====")

```

```

P_Wet_Rain_New = 0.90

```

```

P_WetGrass_New = (
    P_Wet_Rain_New * P_Rain

```

```

+
P_Wet_NoRain * P_NoRain
)

P_Rain_given_Wet_New = (
    P_Wet_Rain_New * P_Rain
) / P_WetGrass_New

print("New P(Wet Grass | Rain) =", P_Wet_Rain_New)
print("New P(Rain | Wet Grass) =", P_Rain_given_Wet_New)
print("\n===== COMPARISON =====")

print("Original P(Rain | Wet Grass) =",
      P_Rain_given_Wet)

print("Modified P(Rain | Wet Grass) =",
      P_Rain_given_Wet_New)

```

## Output

```

===== RESTART: C:/Us
===== WITHOUT EVIDENCE =====
P(Rain) = 0.3
P(No Rain) = 0.7

===== EVIDENCE: WET GRASS =====
P(Wet Grass) = 0.38
P(Rain | Wet Grass) = 0.631578947368421

===== EVIDENCE: DRY GRASS =====
P(Dry Grass) = 0.6199999999999999
P(Rain | Dry Grass) = 0.0967741935483871

===== TRAFFIC =====
P(Traffic) = 0.42

===== AFTER MODIFYING PROBABILITY =====
New P(Wet Grass | Rain) = 0.9
New P(Rain | Wet Grass) = 0.6585365853658537

===== COMPARISON =====
Original P(Rain | Wet Grass) = 0.631578947368421
Modified P(Rain | Wet Grass) = 0.6585365853658537

```



4. **Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.**

**Program :**

```
import numpy as np
```

```
data = [
```

```
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
```

```
    ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
```

```
    ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
```

```
    ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"]
```

```
]
```

```
attributes = [
```

```
    "Sky",
```

```
    "Temperature",
```

```
    "Humidity",
```

```
    "Wind",
```

```
    "Water",
```

```
    "Forecast"
```

```
]
```

```
def candidate_elimination(data):
```

```
    n = len(attributes)
```

```
    S = ["Ø"] * n
```

```
    G = [["?"] * n]
```

```
    for example in data:
```

```
        x = example[:-1]
```

```
        label = example[-1]
```

```
        if label == "Yes":
```

```
            G = [
```

```
                g for g in G
```

```
                if all(g[i] == "?" or g[i] == x[i]
```

```
                    for i in range(n))
```

```
            ]
```

```
        for i in range(n):
```

```
            if S[i] == "Ø":
```

```
                S[i] = x[i]
```

```
            elif S[i] != x[i]:
```

```
                S[i] = "?"
```

else:

new\_G = []

for g in G:

if all(

g[i] == "?" or g[i] == x[i]

for i in range(n)

):

for i in range(n):

if g[i] == "?":

if S[i] != "?" and S[i] != "Ø":

new\_h = g.copy()

new\_h[i] = S[i]

new\_G.append(new\_h)

else:

new\_G.append(g)

G = new\_G

return S, G

S, G = candidate\_elimination(data)

```
print("===== CANDIDATE ELIMINATION =====")
```

```
print("\nSpecific Boundary (S):")
```

```
print(S)
```

```
print("\nGeneral Boundary (G):")
```

```
for hypothesis in G:
```

```
    print(hypothesis)
```

```
print("\n===== EFFECT OF TRAINING SIZE =====")
```

```
for size in range(1, len(data) + 1):
```

```
    S_temp, G_temp = candidate_elimination(data[:size])
```

```
    print("\nTraining Examples:", size)
```

```
    print("S =", S_temp)
```

```
    print("G =", G_temp)
```

**output**

```

===== CANDIDATE ELIMINATION ===== RESTART: C:/Users/HP/OneDrive/Documents/a4.py =====

Specific Boundary (S):
['Sunny', 'Warm', '?', 'Strong', '?', '?']

General Boundary (G):
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
['?', '?', '?', 'Strong', '?', '?']

===== EFFECT OF TRAINING SIZE =====

Training Examples: 1
S = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
G = [['?', '?', '?', '?', '?', '?']]

Training Examples: 2
S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G = [['?', '?', '?', '?', '?', '?']]

Training Examples: 3
S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', 'Strong', '?', '?'], ['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?', '?', 'Same']]

Training Examples: 4
S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', 'Strong', '?', '?']]

```

Ln: 49 Col: 82

5. Design and conduct an experiment to implement a Decision Tree classifier on a real world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations

#### Program :

```

import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

data = load_breast_cancer()

X = pd.DataFrame(
    data.data,
    columns=data.feature_names
)

y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

entropy_tree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)
entropy_tree.fit(X_train, y_train)
y_pred_entropy = entropy_tree.predict(X_test)
print("===== ENTROPY =====")
print("Accuracy:",
      accuracy_score(y_test, y_pred_entropy))
print("F1 Score:",
      f1_score(y_test, y_pred_entropy))
gini_tree = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
gini_tree.fit(X_train, y_train)
y_pred_gini = gini_tree.predict(X_test)
print("\n===== GINI INDEX =====")
print("Accuracy:",
      accuracy_score(y_test, y_pred_gini))
print("F1 Score:",
      f1_score(y_test, y_pred_gini))
print("\n===== TREE DEPTH =====")
for depth in [2, 3, 5, 10, None]:
    tree = DecisionTreeClassifier(
        criterion="gini",
        max_depth=depth,
        random_state=42
    )
    tree.fit(X_train, y_train)

    train_pred = tree.predict(X_train)
    test_pred = tree.predict(X_test)

    train_accuracy = accuracy_score(
        y_train,
        train_pred

```

```

)
test_accuracy = accuracy_score(
    y_test,
    test_pred
)
test_f1 = f1_score(
    y_test,
    test_pred
)
print("\nDepth:", depth)
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy :", test_accuracy)
print("Testing F1 Score :", test_f1)
print("\n===== PRUNING =====")
for alpha in [0.0, 0.001, 0.005, 0.01, 0.02]:
    pruned_tree = DecisionTreeClassifier(
        criterion="gini",
        ccp_alpha=alpha,
        random_state=42
    )
    pruned_tree.fit(X_train, y_train)
    y_pred = pruned_tree.predict(X_test)
    accuracy = accuracy_score(
        y_test,
        y_pred
    )
    f1 = f1_score(
        y_test,
        y_pred
    )
    print("\nCCP Alpha:", alpha)
    print("Accuracy:", accuracy)
    print("F1 Score:", f1)

```

## Output

```
===== RESTART: C:/Users/HP/OneDrive/Documents/a5.py =====  
===== ENTROPY =====  
Accuracy: 0.9122807017543859  
F1 Score: 0.927536231884058  
  
===== GINI INDEX =====  
Accuracy: 0.9122807017543859  
F1 Score: 0.9285714285714286  
  
===== TREE DEPTH =====  
  
Depth: 2  
Training Accuracy: 0.9582417582417583  
Testing Accuracy : 0.8947368421052632  
Testing F1 Score : 0.9117647058823529  
  
Depth: 3  
Training Accuracy: 0.9758241758241758  
Testing Accuracy : 0.9385964912280702  
Testing F1 Score : 0.9517241379310345  
  
Depth: 5  
Training Accuracy: 0.9934065934065934  
Testing Accuracy : 0.9210526315789473  
Testing F1 Score : 0.9361702127659575  
  
Depth: 10  
Training Accuracy: 1.0  
Testing Accuracy : 0.9122807017543859  
Testing F1 Score : 0.9285714285714286  
  
Depth: None  
Training Accuracy: 1.0  
Testing Accuracy : 0.9122807017543859  
Testing F1 Score : 0.9285714285714286
```